

2023 Fall CS 504 Project

Dheeraj Akash Dokuparthi

G01446992

3.1 Database Design

1. Define the scope of the project and identify the entities and their relationships.

Scope of the project

Overview:

The goal of this project is to design and build a database management system for a public library. The system will aid library staff in managing their resources, which will include books, magazines, digital media, and other materials. It will also improve access to library member information and make borrowing and monitoring library books easier. For both library employees and members, the library management system should assure data integrity and reduce redundancy.

Features:

- **Materials Management:** All library materials, including books, magazines, e-books, and audiobooks, should have their names, authors, publication dates, and categories stored and maintained by the system.
- **Membership Management:** Names, contact details, membership numbers, and borrowing histories of library patrons should all be stored and managed by the system.
- **Borrowing:** The system should make it easier for members to check out items and give library employees the data they need to control the circulation of library goods. A librarian should note the borrow date and expected return date of a material once it has been checked out. Additionally, the material's return date needs to be changed after it is returned.
- **Reporting and Analytics:** The system should produce reports on popular resources, library usage, and other pertinent data so that library staff may make informed decisions about resource acquisition and administration.

Entities and Relationships

- Material

Represents individual items available in the library, such as books, magazines, e-books, and audiobooks.

Attributes:

- Material_ID: A unique identifier for each material.
- Title: The title of the material.
- Publication_Date: The date of publication of the material.
- Catalog_ID: A reference to the catalog entry for the material.
- Genre_ID: A reference to the genre of the material.

- Catalog

Represents a record of library materials with information on their availability and location.

Attributes:

- Catalog_ID: A unique identifier for each catalog entry.
- Name: The name of the catalog.
- Location: The location of the material within the library.

- Genre

Represents the various genres or categories of library materials.

Attributes:

- Genre_ID: A unique identifier for each genre.
- Name: The name of the genre.
- Description: The brief introduction of the genre.

- Borrow

Represents the borrowing activity of library materials by members.

Attributes:

- Borrow_ID: A unique identifier for each borrowing transaction.
- Material_ID: A reference to the borrowed material.
- Member_ID: A reference to the member who borrowed the material.
- Staff_ID: A reference to the staff who processed the transaction.
- Borrow_Date: The date the material was borrowed.
- Due_Date: The date the material is due.
- Return_Date: The date the material is returned.

- Author

Represents authors who have created library materials.

Attributes:

- Author_ID: A unique identifier for each author.
- Name: The name of the author.
- Birth_Date: The birth date of the author.
- Nationality: The nationality of the author.

- Authorship

Represents the relationship between authors and the materials they have created.

Attributes:

- Authorship_ID: A unique identifier for each authorship record.
- Author_ID: A reference to the author.
- Material_ID: A reference to the material authored.

- Member

Represents library members who can borrow and reserve materials.

Attributes:

- Member_ID: A unique identifier for each member.
- Name: The name of the member.
- Contact_Info: Email address (or phone number) of the member.
- Join_Date: The date the member joined the library.

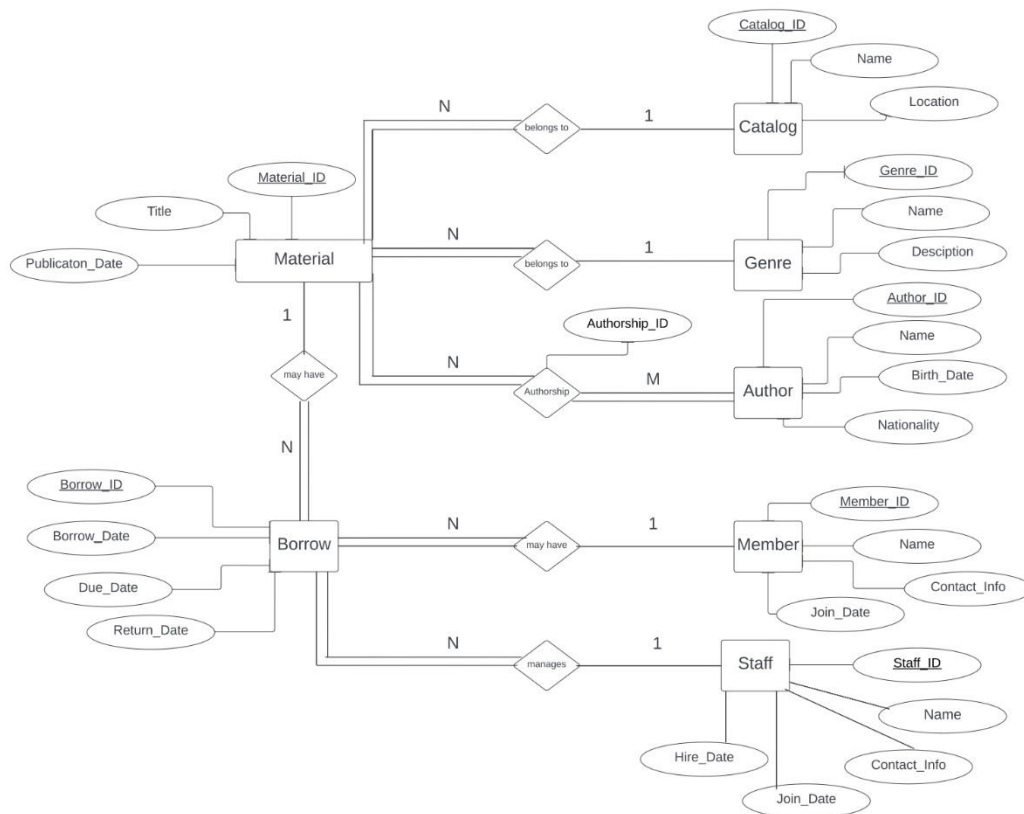
- Staff

Represents library staff who manage library resources and assist members.

Attributes:

- Staff_ID: A unique identifier for each staff member.
- Name: The name of the staff member.
- Contact_Info: Email address (or phone number) of the member.
- Job_Title: The job title of the staff member (e.g., librarian, assistant librarian).
- Hire_Date: The date the staff member was hired by the library

2. Create an Entity-Relationship (ER) diagram to represent the database schema.



3.2 Database Implementation

- Choose an appropriate Database Management System (DBMS) for the project. We recommend using Postgre.
- Implement the database schema using SQL or the chosen DBMS's data definition language (DDL).

--Create a database.

create a database and tables to represent the entities and relationships in the library management system.

-- create db FinProj

```
CREATE DATABASE FinProj;
USE FinProj;
```

-- Create the tables

#Create Catalog table:

```
CREATE TABLE Catalog (  
    Catalog_ID INT PRIMARY KEY,  
    Name VARCHAR(255),  
    Location VARCHAR(255)  
);
```

#Create Genre table:

```
CREATE TABLE Genre (  
    Genre_ID INT PRIMARY KEY,  
    Name VARCHAR(255),  
    Description TEXT  
);
```

#Create Material table:

```
CREATE TABLE Material (  
    Material_ID INT PRIMARY KEY,  
    Title VARCHAR(255),  
    Publication_Date DATE,  
    Catalog_ID INT,  
    Genre_ID INT,  
    FOREIGN KEY (Catalog_ID) REFERENCES Catalog (Catalog_ID),  
    FOREIGN KEY (Genre_ID) REFERENCES Genre (Genre_ID));
```

#Create Author table:

```
CREATE TABLE Author (  
    Author_ID INT PRIMARY KEY,  
    Name VARCHAR(255),  
    Birth_Date DATE,  
    Nationality VARCHAR(255)  
);
```

#Create Authorship table:

```
CREATE TABLE Authorship (  
    Authorship_ID INT PRIMARY KEY,  
    Author_ID INT,  
    Material_ID INT,  
    FOREIGN KEY (Author_ID) REFERENCES Author (Author_ID),  
    FOREIGN KEY (Material_ID) REFERENCES Material (Material_ID));
```

#Create Member table:

```
CREATE TABLE Member (  
    Member_ID INT PRIMARY KEY,
```

```

    Name VARCHAR(255),
    Contact_Info VARCHAR(255),
    Join_Date DATE
);

```

#Create Staff table:

```

CREATE TABLE Staff (
    Staff_ID int PRIMARY KEY,
    Name VARCHAR(255),
    Contact_Info VARCHAR(255),
    Job_Title VARCHAR(255),
    Hire_Date DATE
);

```

#Create Borrow table:

```

CREATE TABLE Borrow (
    Borrow_ID SERIAL PRIMARY KEY,
    Material_ID INT,
    Member_ID INT,
    Staff_ID INT,
    Borrow_Date DATE,
    Due_Date DATE,
    Return_Date DATE,
    FOREIGN KEY (Material_ID) REFERENCES Material (Material_ID),
    FOREIGN KEY (Member_ID) REFERENCES Member (Member_ID),
    FOREIGN KEY (Staff_ID) REFERENCES Staff (Staff_ID)
);

```

- Populate the database with sample data

#Insert data into tables

-- Insert data into Author table

```

INSERT INTO Author (Author_ID, Name, Birth_Date, Nationality) VALUES
(1, 'Jane Austen', '1775-12-16', 'British'),
(2, 'Ernest Hemingway', '1899-07-21', 'American'),
(3, 'George Orwell', '1903-06-25', 'British'),
(4, 'Scott Fitzgerald', '1896-09-24', 'American'),
(5, 'J.K. Rowling', '1965-07-31', 'British'),
(6, 'Mark Twain', '1835-11-30', 'American'),
(7, 'Leo Tolstoy', '1828-09-09', 'Russian'),
(8, 'Virginia Woolf', '1882-01-25', 'British'),
(9, 'Gabriel Márquez', '1927-03-06', 'Colombian'),
(10, 'Charles Dickens', '1812-02-07', 'British'),
(11, 'Harper Lee', '1926-04-28', 'American'),
(12, 'Oscar Wilde', '1854-10-16', 'Irish'),
(13, 'William Shakespeare', '1564-04-26', 'British'),

```

(14, 'Franz Kafka', '1883-07-03', 'Czech'),
 (15, 'James Joyce', '1882-02-02', 'Irish'),
 (16, 'J.R.R. Tolkien', '1892-01-03', 'British'),
 (17, 'Emily Brontë', '1818-07-30', 'British'),
 (18, 'Toni Morrison', '1931-02-18', 'American'),
 (19, 'Fyodor Dostoevsky', '1821-11-11', 'Russian'),
 (20, 'Lucas Piki', '1847-10-16', 'British');

-- Insert data into Catalog table

```
INSERT INTO Catalog (Catalog_ID, Name, Location) VALUES
(1, 'Books', 'A1.1'),
(2, 'Magazines', 'B2.1'),
(3, 'E-Books', 'C3.1'),
(4, 'Audiobooks', 'D4.1'),
(5, 'Journals', 'E5.1'),
(6, 'Newspaper', 'F6.1'),
(7, 'Maps', 'G7.1'),
(8, 'Novels', 'H8.1'),
(9, 'Sheet Music', 'I9.1'),
(10, 'Educational', 'J10.1');
```

-- Insert data into Genre table

```
INSERT INTO Genre (Genre_ID, Name, Description) VALUES
(1, 'General Fiction', 'Literary works with a focus on character and plot development, exploring various themes and human experiences.'),
(2, 'Mystery & Thriller', 'Suspenseful stories centered around crime, investigation, or espionage with an emphasis on tension and excitement.'),
(3, 'Science Fiction & Fantasy', 'Imaginative works that explore alternate realities, futuristic concepts, and magical or supernatural elements.'),
(4, 'Horror & Suspense', 'Stories designed to evoke fear, unease, or dread, often featuring supernatural or psychological elements.'),
(5, 'Dystopian & Apocalyptic', 'Depictions of societies in decline or collapse, often exploring themes of political and social oppression or environmental disaster.'),
(6, 'Classics', 'Enduring works of literature that have stood the test of time, often featuring rich language and complex themes.'),
(7, 'Historical Fiction', 'Fictional stories set in the past, often based on real historical events or figures, and exploring the customs and experiences of that time.'),
(8, 'Epic Poetry & Mythology', 'Ancient or traditional stories and poems, often featuring heroes, gods, and mythical creatures, and exploring cultural values and beliefs.');
```

-- Insert data into Material table

```
INSERT INTO Material (Material_ID, Title, Publication_Date, Catalog_ID,
Genre_ID) VALUES
(1, 'The Catcher in the Rye', '1951-07-16', 1, 1),
(2, 'To Kill a Mockingbird', '1960-07-11', 2, 1),
(3, 'The Da Vinci Code', '2003-04-01', 3, 2),
```

(4, 'The Hobbit', '1937-09-21', 4, 3),
 (5, 'The Shining', '1977-01-28', 5, 4),
 (6, 'Pride and Prejudice', '1813-01-28', 1, 1),
 (7, 'The Great Gatsby', '1925-10-04', 2, 1),
 (8, 'Moby Dick', '1851-10-18', 3, 1),
 (9, 'Crime and Punishment', '1866-01-01', 4, 1),
 (10, 'The Hitchhiker's Guide to the Galaxy', '1979-10-12', 5, 3),
 (11, '1984', '1949-06-08', 1, 5),
 (12, 'Animal Farm', '1945-08-17', 2, 5),
 (13, 'The Haunting of Hill House', '1959-10-17', 3, 4),
 (14, 'Brave New World', '1932-08-01', 4, 5),
 (15, 'The Chronicles of Narnia: The Lion the Witch and the Wardrobe', '1950-10-16', 5, 3),
 (16, 'The Adventures of Huckleberry Finn', '1884-12-10', 6, 1),
 (17, 'The Catch-22', '1961-10-11', 7, 1),
 (18, 'The Picture of Dorian Gray', '1890-07-01', 8, 1),
 (19, 'The Call of Cthulhu', '1928-02-01', 9, 4),
 (20, 'Harry Potter and the Philosopher's Stone', '1997-06-26', 10, 3),
 (21, 'Frankenstein', '1818-01-01', 6, 4),
 (22, 'A Tale of Two Cities', '1859-04-30', 7, 1),
 (23, 'The Iliad', '1750-01-01', 8, 6),
 (24, 'The Odyssey', '1725-01-01', 9, 6),
 (25, 'The Brothers Karamazov', '1880-01-01', 10, 1),
 (26, 'The Divine Comedy', '1320-01-01', 6, 6),
 (27, 'The Grapes of Wrath', '1939-04-14', 7, 1),
 (28, 'The Old Man and the Sea', '1952-09-01', 8, 1),
 (29, 'The Count of Monte Cristo', '1844-01-01', 9, 1),
 (30, 'A Midsummer Night's Dream', '1596-01-01', 10, 7),
 (31, 'The Tricky Book', '1888-01-01', 10, 7);

-- Insert data into Authorship table

INSERT INTO Authorship (Authorship_ID, Author_ID, Material_ID) VALUES

(1, 1, 1),
 (2, 2, 2),
 (3, 3, 3),
 (4, 4, 4),
 (5, 5, 5),
 (6, 6, 6),
 (7, 7, 7),
 (8, 8, 8),
 (9, 9, 9),
 (10, 10, 10),
 (11, 11, 11),
 (12, 12, 12),
 (13, 13, 13),
 (14, 14, 14),
 (15, 15, 15),
 (16, 16, 16),
 (17, 17, 17),

(18, 18, 18),
(19, 19, 19),
(20, 20, 20),
(21, 1, 21),
(22, 2, 22),
(23, 3, 22),
(24, 3, 23),
(25, 4, 24),
(26, 5, 25),
(27, 6, 26),
(28, 7, 27),
(29, 8, 28),
(30, 19, 28),
(31, 9, 29),
(32, 10, 30),
(33, 8, 30),
(34, 2, 29);

-- Insert data into Member table

```
INSERT INTO Member (Member_ID, Name, Contact_Info, Join_Date) VALUES
(1, 'Alice Johnson', 'alice.johnson@email.com', '2018-01-10'),
(2, 'Bob Smith', 'bob.smith@email.com', '2018-03-15'),
(3, 'Carol Brown', 'carol.brown@email.com', '2018-06-20'),
(4, 'David Williams', 'david.williams@email.com', '2018-09-18'),
(5, 'Emily Miller', 'emily.miller@email.com', '2019-02-12'),
(6, 'Frank Davis', 'frank.davis@email.com', '2019-05-25'),
(7, 'Grace Wilson', 'grace.wilson@email.com', '2019-08-15'),
(8, 'Harry Garcia', 'harry.garcia@email.com', '2019-11-27'),
(9, 'Isla Thomas', 'isla.thomas@email.com', '2020-03-04'),
(10, 'Jack Martinez', 'jack.martinez@email.com', '2020-07-01'),
(11, 'Kate Anderson', 'kate.anderson@email.com', '2020-09-30'),
(12, 'Luke Jackson', 'luke.jackson@email.com', '2021-01-18'),
(13, 'Mia White', 'mia.white@email.com', '2021-04-27'),
(14, 'Noah Harris', 'noah.harris@email.com', '2021-07-13'),
(15, 'Olivia Clark', 'olivia.clark@email.com', '2021-10-05'),
(16, 'Peter Lewis', 'peter.lewis@email.com', '2021-12-01'),
(17, 'Quinn Hall', 'quinn.hall@email.com', '2022-02-28'),
(18, 'Rachel Young', 'rachel.young@email.com', '2022-06-17'),
(19, 'Sam Walker', 'sam.walker@email.com', '2022-09-25'),
(20, 'Tiffany Allen', 'tiffany.allen@email.com', '2022-12-10');
```

-- Insert data into Staff table

```
INSERT INTO Staff (Staff_ID, Name, Contact_Info, Job_Title, Hire_Date) VALUES
(1, 'Amy Green', 'amy.green@email.com', 'Librarian', '2017-06-01'),
(2, 'Brian Taylor', 'brian.taylor@email.com', 'Library Assistant', '2018-11-15'),
(3, 'Christine King', 'chris.king@email.com', 'Library Assistant', '2019-05-20'),
(4, 'Daniel Wright', 'dan.wright@email.com', 'Library Technician', '2020-02-01');
```

-- Insert data into Borrow table

```
INSERT INTO Borrow (Borrow_ID, Material_ID, Member_ID, Staff_ID,
Borrow_Date, Due_Date, Return_Date) VALUES
(1, 1, 1, 1, '2018-09-12', '2018-10-03', '2018-09-30'),
(2, 2, 2, 1, '2018-10-15', '2018-11-05', '2018-10-29'),
(3, 3, 3, 1, '2018-12-20', '2019-01-10', '2019-01-08'),
(4, 4, 4, 1, '2019-03-11', '2019-04-01', '2019-03-27'),
(5, 5, 5, 1, '2019-04-20', '2019-05-11', '2019-05-05'),
(6, 6, 6, 1, '2019-07-05', '2019-07-26', '2019-07-21'),
(7, 7, 7, 1, '2019-09-10', '2019-10-01', '2019-09-25'),
(8, 8, 8, 1, '2019-11-08', '2019-11-29', '2019-11-20'),
(9, 9, 9, 1, '2020-01-15', '2020-02-05', '2020-02-03'),
(10, 10, 10, 1, '2020-03-12', '2020-04-02', '2020-03-28'),
(11, 1, 11, 2, '2020-05-14', '2020-06-04', '2020-05-28'),
(12, 2, 12, 2, '2020-07-21', '2020-08-11', '2020-08-02'),
(13, 3, 13, 2, '2020-09-25', '2020-10-16', '2020-10-15'),
(14, 4, 1, 2, '2020-11-08', '2020-11-29', '2020-11-24'),
(15, 5, 2, 2, '2021-01-03', '2021-01-24', '2021-01-19'),
(16, 6, 3, 2, '2021-02-18', '2021-03-11', '2021-03-12'),
(17, 17, 4, 2, '2021-04-27', '2021-05-18', '2021-05-20'),
(18, 18, 5, 2, '2021-06-13', '2021-07-04', '2021-06-28'),
(19, 19, 6, 2, '2021-08-15', '2021-09-05', '2021-09-03'),
(20, 20, 7, 2, '2021-10-21', '2021-11-11', NULL),
(21, 21, 1, 3, '2021-11-29', '2021-12-20', NULL),
(22, 22, 2, 3, '2022-01-10', '2022-01-31', '2022-01-25'),
(23, 23, 3, 3, '2022-02-07', '2022-02-28', '2022-02-23'),
(24, 24, 4, 3, '2022-03-11', '2022-04-01', '2022-03-28'),
(25, 25, 5, 3, '2022-04-28', '2022-05-19', '2022-05-18'),
(26, 26, 6, 3, '2022-06-22', '2022-07-13', '2022-07-08'),
(27, 27, 7, 3, '2022-08-04', '2022-08-25', '2022-08-23'),
(28, 28, 8, 3, '2022-09-13', '2022-10-04', '2022-09-28'),
(29, 29, 9, 3, '2022-10-16', '2022-11-06', '2022-11-05'),
(30, 30, 8, 3, '2022-11-21', '2022-12-12', '2022-12-05'),
(31, 1, 9, 4, '2022-12-28', '2023-01-18', NULL),
(32, 2, 1, 4, '2023-01-23', '2023-02-13', NULL),
(33, 3, 10, 4, '2023-02-02', '2023-02-23', '2023-02-17'),
(34, 4, 11, 4, '2023-03-01', '2023-03-22', NULL),
(35, 5, 12, 4, '2023-03-10', '2023-03-31', NULL),
(36, 6, 13, 4, '2023-03-15', '2023-04-05', NULL),
(37, 7, 17, 4, '2023-03-25', '2023-04-15', NULL),
(38, 8, 8, 4, '2023-03-30', '2023-04-20', NULL),
(39, 9, 9, 4, '2023-03-26', '2023-04-16', NULL),
(40, 10, 20, 4, '2023-03-28', '2023-04-18', NULL);
```

3.3 Querying and Manipulation

These below example queries are executed after the “4.1 Queries/Updates executions”.

1. Develop a set of SQL queries or stored procedures to perform common tasks, such as searching, updating, inserting, and deleting records.

-- **Searching example:** Search for Members by Name like bob.

```
SELECT *  
FROM member  
WHERE Name LIKE '%Bob%';
```

Result:

The screenshot shows a SQL query window with the following text:

```
361 -- Searching example: Search for Members by Name  
362 • SELECT *  
363 FROM member  
364 WHERE Name LIKE '%Bob%';  
365  
366  
367  
368  
369  
370
```

Below the query window is the 'Result Grid' showing the results of the query:

Member_ID	Name	Contact_Info	Join_Date
2	Bob Smith	bob.smith@email.com	2018-03-15
NULL	NULL	NULL	NULL

At the bottom, the 'Output' window shows the 'Action Output' for the query:

#	Time	Action	Message
1	12:23:05	USE FinProj	0 row(s) affected

-- **Inserting example:** Add a new member:

```
INSERT INTO Member (Member_ID, Name, Contact_Info, Join_Date) VALUES  
(999, 'Ali John', 'ali.john@email.com', '2023-08-12');
```

Result:

The screenshot shows a SQL query window with the following text:

```
366 -- Inserting Records:  
367 -- Add a New Member:  
368 • INSERT INTO Member (Member_ID, Name, Contact_Info, Join_Date) VALUES  
369 (999, 'Ali John', 'ali.john@email.com', '2023-08-12');  
370  
371  
372  
373  
374  
375
```

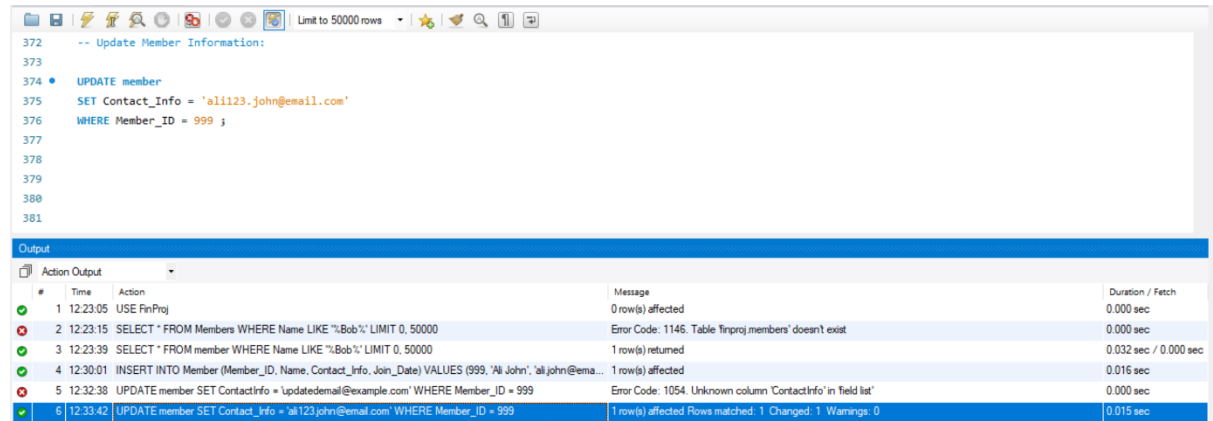
Below the query window is the 'Output' window showing the 'Action Output' for the query:

#	Time	Action	Message	Duration / Fetch
1	12:23:05	USE FinProj	0 row(s) affected	0.000 sec
2	12:23:15	SELECT * FROM Members WHERE Name LIKE '%Bob%' LIMIT 0, 50000	Error Code: 1146. Table 'finproj.members' doesn't exist	0.000 sec
3	12:23:39	SELECT * FROM member WHERE Name LIKE '%Bob%' LIMIT 0, 50000	1 row(s) returned	0.032 sec / 0.000 sec
4	12:30:01	INSERT INTO Member (Member_ID, Name, Contact_Info, Join_Date) VALUES (999, 'Ali John', 'ali.john@email.com', '2023-08-12');	1 row(s) affected	0.016 sec

-- **Example for updating a record:** Update member table Information:

```
UPDATE member
SET Contact_Info = 'ali123.john@email.com'
WHERE Member_ID = 999 ;
```

Result:



The screenshot shows a database client interface with a SQL editor and an output window. The SQL editor contains the following code:

```
-- Update Member Information:
372
373
374 UPDATE member
375 SET Contact_Info = 'ali123.john@email.com'
376 WHERE Member_ID = 999 ;
377
378
379
380
381
```

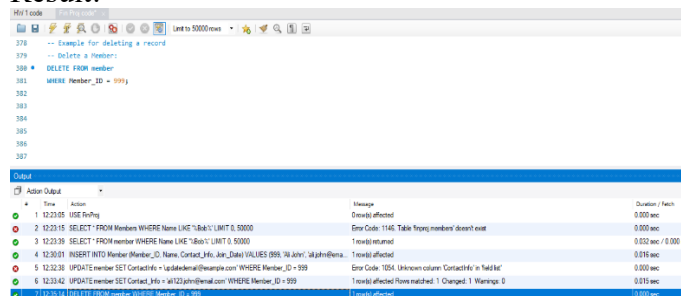
The output window displays the execution results for the UPDATE statement. The table below summarizes the data shown in the screenshot:

#	Time	Action	Message	Duration / Fetch
1	12:23:05	USE FinProj	0 row(s) affected	0.000 sec
2	12:23:15	SELECT * FROM Members WHERE Name LIKE '%Bob%' LIMIT 0, 50000	Error Code: 1146. Table 'finproj.members' doesn't exist	0.000 sec
3	12:23:39	SELECT * FROM member WHERE Name LIKE '%Bob%' LIMIT 0, 50000	1 row(s) returned	0.032 sec / 0.000 sec
4	12:30:01	INSERT INTO Member (Member_ID, Name, Contact_Info, Join_Date) VALUES (999, 'Ali John', 'ali.john@ema...	1 row(s) affected	0.016 sec
5	12:32:38	UPDATE member SET ContactInfo = 'Updatedemail@example.com' WHERE Member_ID = 999	Error Code: 1054. Unknown column 'ContactInfo' in 'field list'	0.000 sec
6	12:33:42	UPDATE member SET Contact_Info = 'ali123.john@email.com' WHERE Member_ID = 999	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec

-- **Example for deleting a record:** Delete a member:

```
DELETE FROM member
WHERE Member_ID = 999;
```

Result:



The screenshot shows a database client interface with a SQL editor and an output window. The SQL editor contains the following code:

```
-- Example for deleting a record
378 -- Delete a Member:
379
380 DELETE FROM member
381 WHERE Member_ID = 999;
382
383
384
385
386
387
```

The output window displays the execution results for the DELETE statement. The table below summarizes the data shown in the screenshot:

#	Time	Action	Message	Duration / Fetch
1	12:23:05	USE FinProj	0 row(s) affected	0.000 sec
2	12:23:15	SELECT * FROM Members WHERE Name LIKE '%Bob%' LIMIT 0, 50000	Error Code: 1146. Table 'finproj.members' doesn't exist	0.000 sec
3	12:23:39	SELECT * FROM member WHERE Name LIKE '%Bob%' LIMIT 0, 50000	1 row(s) returned	0.032 sec / 0.000 sec
4	12:30:01	INSERT INTO Member (Member_ID, Name, Contact_Info, Join_Date) VALUES (999, 'Ali John', 'ali.john@ema...	1 row(s) affected	0.016 sec
5	12:32:38	UPDATE member SET ContactInfo = 'Updatedemail@example.com' WHERE Member_ID = 999	Error Code: 1054. Unknown column 'ContactInfo' in 'field list'	0.000 sec
6	12:33:42	UPDATE member SET Contact_Info = 'ali123.john@email.com' WHERE Member_ID = 999	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec
7	12:33:14	DELETE FROM member WHERE Member_ID = 999	1 row(s) affected	0.000 sec

2. Demonstrate the use of advanced querying techniques, such as joins, aggregation, and subqueries

-- **Advanced Queries:**

-- **Find the top 5 authors with the most published materials:**

```
SELECT Author.Name, COUNT(Authorship.Material_ID) AS PublishedMaterials
FROM Author
LEFT JOIN Authorship ON Author.Author_ID = Authorship.Author_ID
GROUP BY Author.Name
ORDER BY PublishedMaterials DESC
LIMIT 5;
```

Result:

The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```

383 -- Advanced Queries:
384 -- Using LeftJoin, Groupby, Orderby.
385 -- Find the top 5 authors with the most published materials:
386 SELECT Author.Name, COUNT(Authorship.Material_ID) AS PublishedMaterials
387 FROM Author
388 LEFT JOIN Authorship ON Author.Author_ID = Authorship.Author_ID
389 GROUP BY Author.Name
390 ORDER BY PublishedMaterials DESC
391 LIMIT 5;

```

The results are displayed in a table with the following data:

Name	PublishedMaterials
Ernest Hemingway	3
George Orwell	3
Virginia Woolf	3
Jane Austen	2
J.K. Rowling	2

The bottom of the screenshot shows the output log with the following entries:

#	Time	Action	Message	Duration / Fetch
6	12:33:42	UPDATE member SET Contact_Info = 'all123john@gmail.com' WHERE Member_ID = 999	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec
7	12:35:14	DELETE FROM member WHERE Member_ID = 999	1 row(s) affected	0.000 sec
8	12:50:09	SELECT Author Name, COUNT(Authorship.Material_ID) AS PublishedMaterials FROM Author LEFT JOIN A	5 row(s) returned	0.016 sec / 0.000 sec

-- Retrieve the names of staff members and their job titles for materials published before a specific date:

```

SELECT DISTINCT Staff.Name, Staff.Job_Title
FROM Staff
INNER JOIN Borrow ON Staff.Staff_ID = Borrow.Staff_ID
INNER JOIN Material ON Borrow.Material_ID = Material.Material_ID
WHERE Material.Publication_Date < '2000-01-01';

```

Result:

The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```

392 -- Retrieve the names of staff members and their job titles for materials published before a specific date:
393 SELECT DISTINCT Staff.Name, Staff.Job_Title
394 FROM Staff
395 INNER JOIN Borrow ON Staff.Staff_ID = Borrow.Staff_ID
396 INNER JOIN Material ON Borrow.Material_ID = Material.Material_ID
397 WHERE Material.Publication_Date < '2000-01-01';
398

```

The results are displayed in a table with the following data:

Name	Job_Title
Amey Green	Librarian
Brian Taylor	Library Assistant
Christine King	Library Assistant
Daniel Wright	Library Technician

The bottom of the screenshot shows the output log with the following entries:

#	Time	Action	Message	Duration / Fetch
7	12:35:14	DELETE FROM member WHERE Member_ID = 999	1 row(s) affected	0.000 sec
8	12:50:09	SELECT Author Name, COUNT(Authorship.Material_ID) AS PublishedMaterials FROM Author LEFT JOIN A	5 row(s) returned	0.015 sec / 0.000 sec
9	12:52:49	SELECT DISTINCT Staff Name, Staff Job_Title FROM Staff INNER JOIN Borrow ON Staff.Staff_ID = Borrow	4 row(s) returned	0.016 sec / 0.000 sec

-- Aggregation and Subquery Example:

-- Find the Most Popular Genre and Its Most Popular Material:

-- Step 1: Find the most popular genre (genre with the highest number of materials)

```

SELECT Genre.Name AS MostPopularGenre, COUNT(Material.Material_ID) AS
MaterialCount
FROM Genre

```

```

INNER JOIN Material ON Genre.Genre_ID = Material.Genre_ID
GROUP BY Genre.Name
ORDER BY MaterialCount DESC
LIMIT 1;
Result:

```

The screenshot shows a SQL IDE with a query window and a results grid. The query is as follows:

```

397 WHERE Material.Publication_Date < '2009-01-01';
398
399 -- Aggregation and Subquery Example:
400 -- Find the most Popular Genre and its Most Popular Material:
401 -- Step 1: Find the most popular genre (genre with the highest number of materials)
402 SELECT Genre.Name AS MostPopularGenre, COUNT(Material.Material_ID) AS MaterialCount
403 FROM Genre
404 INNER JOIN Material ON Genre.Genre_ID = Material.Genre_ID
405 GROUP BY Genre.Name
406 ORDER BY MaterialCount DESC
407 LIMIT 1;
408

```

The results grid shows the following data:

MostPopularGenre	MaterialCount
General Fiction	14

The output window shows the execution of the query and its results:

#	Time	Action	Message	Duration / Fetch
8	12:50:09	SELECT Author Name, COUNT(Authorship.Material_ID) AS PublishedMaterials FROM Author LEFT JOIN A...	5 row(s) returned	0.016 sec / 0.000 sec
9	12:52:49	SELECT DISTINCT Staff Name, Staff Job Title FROM Staff INNER JOIN Borrow ON Staff Staff_ID = Boro...	4 row(s) returned	0.016 sec / 0.000 sec
10	12:55:31	SELECT Material Title AS MostPopularMaterial, Author Name AS Author FROM Material INNER JOIN Genre...	17 row(s) returned	0.016 sec / 0.000 sec
11	12:55:49	SELECT Genre Name AS MostPopularGenre, COUNT(Material.Material_ID) AS MaterialCount FROM Genre...	1 row(s) returned	0.000 sec / 0.000 sec
12	12:56:05	SELECT Material Title AS MostPopularMaterial, Author Name AS Author FROM Material INNER JOIN Genre...	17 row(s) returned	0.000 sec / 0.000 sec
13	13:36:10	SELECT Genre Name AS MostPopularGenre, COUNT(Material.Material_ID) AS MaterialCount FROM Genre...	1 row(s) returned	0.000 sec / 0.000 sec

-- Step 2: Find the most popular material in the most popular genre

```

SELECT Material.Title AS MostPopularMaterial, Author.Name AS Author
FROM Material
INNER JOIN Genre ON Material.Genre_ID = Genre.Genre_ID
INNER JOIN Authorship ON Material.Material_ID = Authorship.Material_ID
INNER JOIN Author ON Authorship.Author_ID = Author.Author_ID
WHERE Genre.Genre_ID = (
    SELECT Genre.Genre_ID
    FROM Genre
    INNER JOIN Material ON Genre.Genre_ID = Material.Genre_ID
    GROUP BY Genre.Genre_ID
    ORDER BY COUNT(Material.Material_ID) DESC
    LIMIT 1
);
Result:

```

The screenshot shows a SQL IDE with a query window and a results grid. The query is as follows:

```

409 -- Step 2: Find the most popular material in the most popular genre
410 SELECT Material.Title AS MostPopularMaterial, Author.Name AS Author
411 FROM Material
412 INNER JOIN Genre ON Material.Genre_ID = Genre.Genre_ID
413 INNER JOIN Authorship ON Material.Material_ID = Authorship.Material_ID
414 INNER JOIN Author ON Authorship.Author_ID = Author.Author_ID
415 WHERE Genre.Genre_ID = (
416     SELECT Genre.Genre_ID
417     FROM Genre
418     INNER JOIN Material ON Genre.Genre_ID = Material.Genre_ID
419     GROUP BY Genre.Genre_ID
420     ORDER BY COUNT(Material.Material_ID) DESC
421     LIMIT 1
422 );
423

```

The results grid shows the following data:

MostPopularMaterial	Author
The Catcher in the Rye	Jane Austen
To Kill a Mockingbird	Ernest Hemingway
Pride and Prejudice	Mark Twain
The Great Gatsby	Leo Tolstoy
Hoby Dick	Virginia Woolf
Crime and Punishment	Gabriel Márquez
The Adventures of Huckleberry Finn	J.R.R. Tolkien
The Catch-22	Emily Brontë
The Picture of Dorian Gray	Toni Morrison
A Tale of Two Cities	Ernest Hemingway

The output window shows the execution of the query and its results:

#	Time	Action	Message	Duration / Fetch
14	13:37:18	SELECT Genre Name AS MostPopularGenre, COUNT(Material.Material_ID) AS MaterialCount FROM Genre...	1 row(s) returned	0.000 sec / 0.000 sec
15	13:37:29	SELECT Material Title AS MostPopularMaterial, Author Name AS Author FROM Material INNER JOIN Genre...	17 row(s) returned	0.000 sec / 0.000 sec

4.1. Queries/Updates

1. Which materials are currently available in the library? If a material is borrowed and not returned, it's not considered as available.

Query:

```
SELECT DISTINCT material. Material_ID, material. Title
FROM Material
WHERE material. Material_ID NOT IN (
SELECT Material_ID
FROM Borrow
WHERE Return_Date IS NULL
);
```

Result:

The screenshot shows a database query interface. The top section displays the SQL query: `SELECT DISTINCT material. Material_ID, material. Title FROM Material WHERE material. Material_ID NOT IN (SELECT Material_ID FROM Borrow WHERE Return_Date IS NULL);`. Below the query, the 'Result Grid' shows a table with two columns: 'Material_ID' and 'Title'. The table contains 12 rows of data, including titles like 'The Da Vinci Code', '1984', 'Animal Farm', 'The Hunting of Hill House', 'Brave New World', 'The Chronicles of Narnia: The Lion the Witch and the Wardrobe', 'The Adventures of Huckleberry Finn', 'The Catch-22', 'The Picture of Dorian Gray', 'The Call of Cthulhu', 'Harry Potter and the Philosopher's Stone', and 'A Tale of Two Cities'. The bottom section shows the 'Output' pane with a table of actions and their results. The first action is 'USE FinProj' with 0 rows affected. The second action is 'SELECT DISTINCT material. Material_ID, material. Title FROM Material WHERE material. Material_ID NOT IN ...' which returned 22 rows in 0.031 seconds.

Material_ID	Title
3	The Da Vinci Code
11	1984
12	Animal Farm
13	The Hunting of Hill House
14	Brave New World
15	The Chronicles of Narnia: The Lion the Witch an...
16	The Adventures of Huckleberry Finn
17	The Catch-22
18	The Picture of Dorian Gray
19	The Call of Cthulhu
20	Harry Potter and the Philosopher's Stone
22	A Tale of Two Cities

#	Time	Action	Message	Duration / Fetch
2	15:32:03	USE FinProj	0 row(s) affected	0.000 sec
3	15:33:07	SELECT DISTINCT material. Material_ID, material. Title FROM Material WHERE material. Material_ID NOT IN ...	22 row(s) returned	0.031 sec / 0.000 sec

2. Which materials are currently overdue? Suppose today is 04/01/2023, and show the borrow date and due date of each material.

Query:

```
SELECT M.Material_ID, M.Title, B.Borrow_Date, B.Due_Date
FROM Material M
INNER JOIN Borrow B ON M.Material_ID = B.Material_ID
WHERE B.Due_Date < '2023-04-01' AND B.Return_Date IS NULL;
```

Result:

```
271 -- 2. Which materials are currently overdue? Suppose today is 04/01/2023, and show the borrow date and due date of each material.
272 SELECT M.Material_ID, M.Title, B.Borrow_Date, B.Due_Date
273 FROM Material M
274 INNER JOIN Borrow B ON M.Material_ID = B.Material_ID
275 WHERE B.Due_Date < '2023-04-01' AND B.Return_Date IS NULL;
276
```

Material_ID	Title	Borrow_Date	Due_Date
20	Harry Potter and the Philosopher's Stone	2021-10-21	2021-11-11
21	Frankenstein	2021-11-29	2021-12-30
1	The Catcher in the Rye	2022-12-28	2023-01-18
2	To Kill a Mockingbird	2023-01-23	2023-02-13
4	The Hobbit	2023-03-01	2023-03-22
5	The Shining	2023-03-10	2023-03-31

Result 2 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
31	13:43:51	SELECT M.Material_ID, M.Title FROM Material M LEFT JOIN Borrow B ON M.Material_ID = B.Material_ID	18 row(s) returned	0.000 sec / 0.000 sec
32	13:45:22	SELECT M.Material_ID, M.Title, B.Borrow_Date, B.Due_Date FROM Material M INNER JOIN Borrow B ON ...	6 row(s) returned	0.016 sec / 0.000 sec

3. What are the top 10 most borrowed materials in the library? Show the title of each material and order them based on their available counts.

Query:

```
SELECT M.Material_ID, M.Title, COUNT(*) AS BorrowCount
FROM Material M
INNER JOIN Borrow B ON M.Material_ID = B.Material_ID
GROUP BY M.Material_ID, M.Title
ORDER BY BorrowCount DESC
LIMIT 10;
```

Result:

```
277 -- 3. What are the top 10 most borrowed materials in the library? Show the title of each material and order them based on their available counts.
278 SELECT M.Material_ID, M.Title, COUNT(*) AS BorrowCount
279 FROM Material M
280 INNER JOIN Borrow B ON M.Material_ID = B.Material_ID
281 GROUP BY M.Material_ID, M.Title
282 ORDER BY BorrowCount DESC
283 LIMIT 10;
284
285
```

Material_ID	Title	BorrowCount
1	The Catcher in the Rye	3
2	To Kill a Mockingbird	3
3	The Da Vinci Code	3
4	The Hobbit	3
5	The Shining	3
6	Pride and Prejudice	3
7	The Great Gatsby	2
8	Hoby Dick	2
9	Crime and Punishment	2
10	The Hitchhiker's Guide to the Galaxy	2

Result 3 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
32	13:45:22	SELECT M.Material_ID, M.Title, B.Borrow_Date, B.Due_Date FROM Material M INNER JOIN Borrow B ON ...	6 row(s) returned	0.016 sec / 0.000 sec
33	13:47:52	SELECT M.Material_ID, M.Title, COUNT(*) AS BorrowCount FROM Material M INNER JOIN Borrow B ON ...	10 row(s) returned	0.000 sec / 0.000 sec

4. How many materials has the author Lucas Piki written?

Query:

```
SELECT COUNT(*) AS TotalMaterialsWritten
FROM Author A
INNER JOIN Authorship ASH ON A.Author_ID = ASH.Author_ID
WHERE A.Name = 'Lucas Piki';
```

Result:

The screenshot shows a database query editor with the following SQL query:

```
-- 4. How many materials has the author Lucas Piki written?
SELECT COUNT(*) AS TotalMaterialsWritten
FROM Author A
INNER JOIN Authorship ASH ON A.Author_ID = ASH.Author_ID
WHERE A.Name = 'Lucas Piki';
```

The results pane shows a single row with the value 1 for TotalMaterialsWritten.

#	Time	Action	Message	Duration / Fetch
33	13:47:52	SELECT M.Material_ID, M.Title, COUNT(*) AS BorrowCount FROM Material M INNER JOIN Borrow B ON ...	10 row(s) returned	0.000 sec / 0.000 sec
34	13:48:24	SELECT COUNT(*) AS TotalMaterialsWritten FROM Author A INNER JOIN Authorship ASH ON A.Author_ID = ...	1 row(s) returned	0.016 sec / 0.000 sec

5. How many materials were written by two or more authors?

Query:

```
SELECT COUNT(*) AS TotalMaterials
FROM (
    SELECT Material_ID, COUNT(*) AS AuthorCount
    FROM Authorship
    GROUP BY Material_ID
    HAVING AuthorCount >= 2
) AS MultiAuthorMaterials;
```

Result:

The screenshot shows a database query editor with the following SQL query:

```
-- 5. How many materials were written by two or more authors?
SELECT COUNT(*) AS TotalMaterials
FROM (
    SELECT Material_ID, COUNT(*) AS AuthorCount
    FROM Authorship
    GROUP BY Material_ID
    HAVING AuthorCount >= 2
) AS MultiAuthorMaterials;
```

The results pane shows a single row with the value 4 for TotalMaterials.

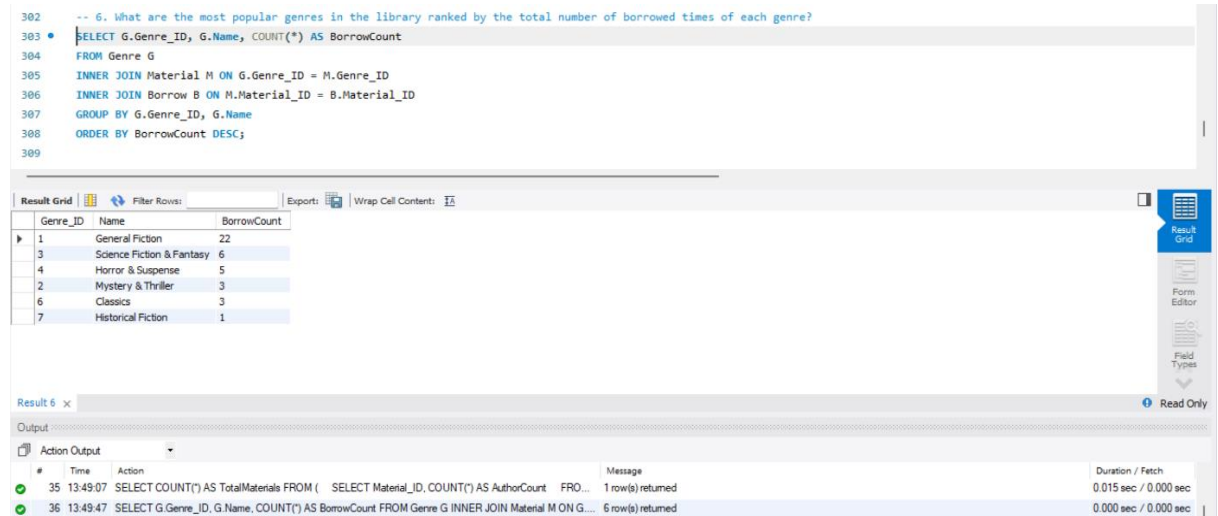
#	Time	Action	Message	Duration / Fetch
34	13:48:24	SELECT COUNT(*) AS TotalMaterialsWritten FROM Author A INNER JOIN Authorship ASH ON A.Author_ID = ...	1 row(s) returned	0.016 sec / 0.000 sec
35	13:49:07	SELECT COUNT(*) AS TotalMaterials FROM (SELECT Material_ID, COUNT(*) AS AuthorCount FROM ...	1 row(s) returned	0.015 sec / 0.000 sec

6. What are the most popular genres in the library ranked by the total number of borrowed times of each genre?

Query:

```
SELECT G.Genre_ID, G.Name, COUNT(*) AS BorrowCount
FROM Genre G
INNER JOIN Material M ON G.Genre_ID = M.Genre_ID
INNER JOIN Borrow B ON M.Material_ID = B.Material_ID
GROUP BY G.Genre_ID, G.Name
ORDER BY BorrowCount DESC;
```

Result:



The screenshot shows a database query editor with the following SQL query:

```
-- 6. What are the most popular genres in the library ranked by the total number of borrowed times of each genre?
SELECT G.Genre_ID, G.Name, COUNT(*) AS BorrowCount
FROM Genre G
INNER JOIN Material M ON G.Genre_ID = M.Genre_ID
INNER JOIN Borrow B ON M.Material_ID = B.Material_ID
GROUP BY G.Genre_ID, G.Name
ORDER BY BorrowCount DESC;
```

The results are displayed in a grid with the following data:

Genre_ID	Name	BorrowCount
1	General Fiction	22
3	Science Fiction & Fantasy	6
4	Horror & Suspense	5
2	Mystery & Thriller	3
6	Classics	3
7	Historical Fiction	1

The output pane shows the following message:

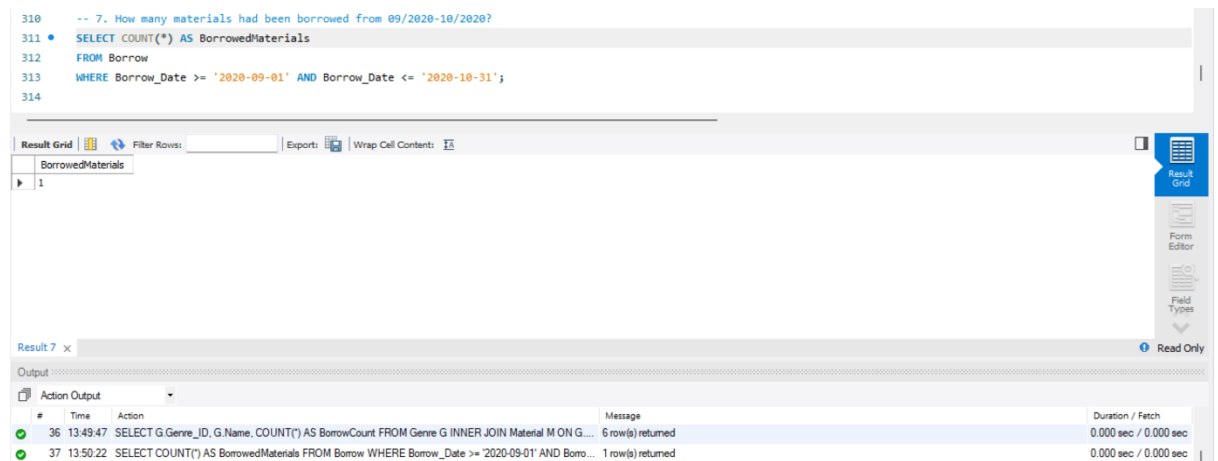
```
35 13:49:07 SELECT COUNT(*) AS TotalMaterials FROM ( SELECT Material_ID, COUNT(*) AS AuthorCount FROM ... 1 row(s) returned
36 13:49:47 SELECT G.Genre_ID, G.Name, COUNT(*) AS BorrowCount FROM Genre G INNER JOIN Material M ON G... 6 row(s) returned
```

7. How many materials had been borrowed from 09/2020-10/2020?

Query:

```
SELECT COUNT(*) AS BorrowedMaterials
FROM Borrow
WHERE Borrow_Date >= '2020-09-01' AND Borrow_Date <= '2020-10-31';
```

Result:



The screenshot shows a database query editor with the following SQL query:

```
-- 7. How many materials had been borrowed from 09/2020-10/2020?
SELECT COUNT(*) AS BorrowedMaterials
FROM Borrow
WHERE Borrow_Date >= '2020-09-01' AND Borrow_Date <= '2020-10-31';
```

The results are displayed in a grid with the following data:

BorrowedMaterials
1

The output pane shows the following message:

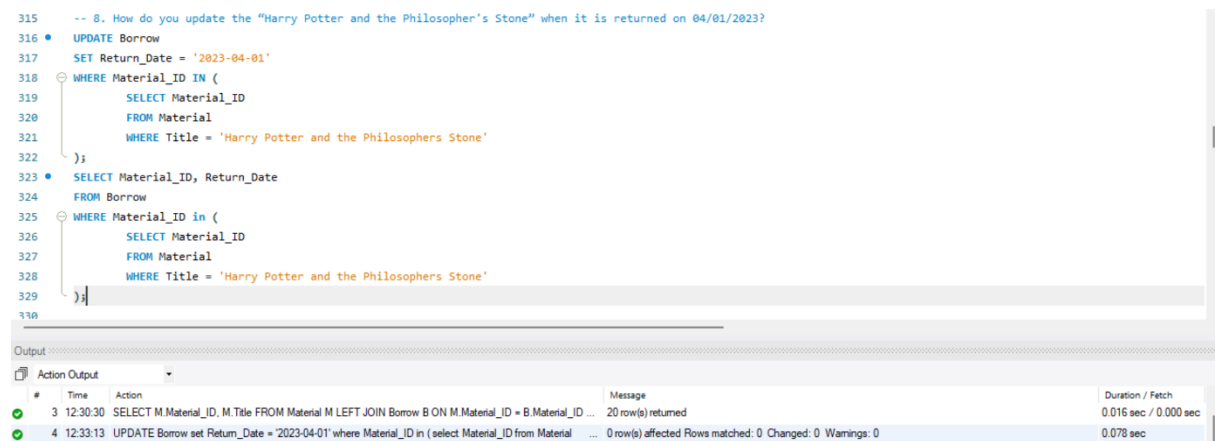
```
36 13:49:47 SELECT G.Genre_ID, G.Name, COUNT(*) AS BorrowCount FROM Genre G INNER JOIN Material M ON G... 6 row(s) returned
37 13:50:22 SELECT COUNT(*) AS BorrowedMaterials FROM Borrow WHERE Borrow_Date >= '2020-09-01' AND Boro... 1 row(s) returned
```

8. How do you update the “Harry Potter and the Philosopher's Stone” when it is returned on 04/01/2023?

Query:

```
UPDATE Borrow
SET Return_Date = '2023-04-01'
WHERE Material_ID IN (
    SELECT Material_ID
    FROM Material
    WHERE Title = 'Harry Potter and the Philosophers Stone'
);
SELECT Material_ID, Return_Date
FROM Borrow
WHERE Material_ID in (
    SELECT Material_ID
    FROM Material
    WHERE Title = 'Harry Potter and the Philosophers Stone'
);
```

Result:



The screenshot shows a SQL IDE with a query editor and an output window. The query editor contains the following SQL code:

```
-- 8. How do you update the "Harry Potter and the Philosopher's Stone" when it is returned on 04/01/2023?
UPDATE Borrow
SET Return_Date = '2023-04-01'
WHERE Material_ID IN (
    SELECT Material_ID
    FROM Material
    WHERE Title = 'Harry Potter and the Philosophers Stone'
);
SELECT Material_ID, Return_Date
FROM Borrow
WHERE Material_ID in (
    SELECT Material_ID
    FROM Material
    WHERE Title = 'Harry Potter and the Philosophers Stone'
);
```

The output window shows the results of the query execution. It includes a table with columns: #, Time, Action, Message, and Duration / Fetch. The table contains two rows of data.

#	Time	Action	Message	Duration / Fetch
3	12:30:30	SELECT M.Material_ID, M.Title FROM Material M LEFT JOIN Borrow B ON M.Material_ID = B.Material_ID ...	20 row(s) returned	0.016 sec / 0.000 sec
4	12:33:13	UPDATE Borrow set Return_Date = '2023-04-01' where Material_ID in (select Material_ID from Material ...	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0	0.078 sec

9. How do you delete the member Emily Miller and all her related records from the database?

Query:

```
-- Delete Borrow records related to "Emily Miller":
DELETE FROM Borrow
WHERE Member_ID = (SELECT Member_ID FROM Member WHERE Name =
'Emily Miller');

-- Delete the "Emily Miller" member record:
SET SQL_SAFE_UPDATES = 0;
DELETE FROM Member
```

```
WHERE Name = 'Emily Miller';
SET SQL_SAFE_UPDATES = 1;
```

Result:

```
334 -- 9. How do you delete the member Emily Miller and all her related records from the database?
335 -- Delete Borrow records related to "Emily Miller":
336 • DELETE FROM Borrow
337 WHERE Member_ID = (SELECT Member_ID FROM Member WHERE Name = 'Emily Miller');
338
339 -- Delete the "Emily Miller" member record:
340 • SET SQL_SAFE_UPDATES = 0;
341 • DELETE FROM Member
342 WHERE Name = 'Emily Miller';
343 • SET SQL_SAFE_UPDATES = 1;
344 |
```

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
7	15:06:06	DELETE FROM Borrow WHERE Member_ID = (SELECT Member_ID FROM Member WHERE Name = 'Emily Miller');	0 row(s) affected	0.016 sec
8	15:06:44	DELETE FROM Member WHERE Name = 'Emily Miller';	0 row(s) affected	0.000 sec

10. How do you add the following material to the database?

Title: New book

Date: 2020-08-01

Catalog: E-Books

Genre: Mystery & Thriller

Author: Lucas Luke

Query:

-- Insert the "New book" into the Material table:

```
INSERT INTO Material (Material_ID, Title, Publication_Date, Catalog_ID,
Genre_ID)
VALUES (32, 'New book', '2020-08-01', (SELECT Catalog_ID FROM Catalog
WHERE Name = 'E-Books'), (SELECT Genre_ID FROM Genre WHERE Name =
'Mystery & Thriller'));
```

-- Insert the Author "Lucas Luke" into the Author table:

```
INSERT INTO Author (Author_ID, Name, Birth_Date, Nationality)
VALUES (21, 'Lucas Luke', NULL, NULL);
```

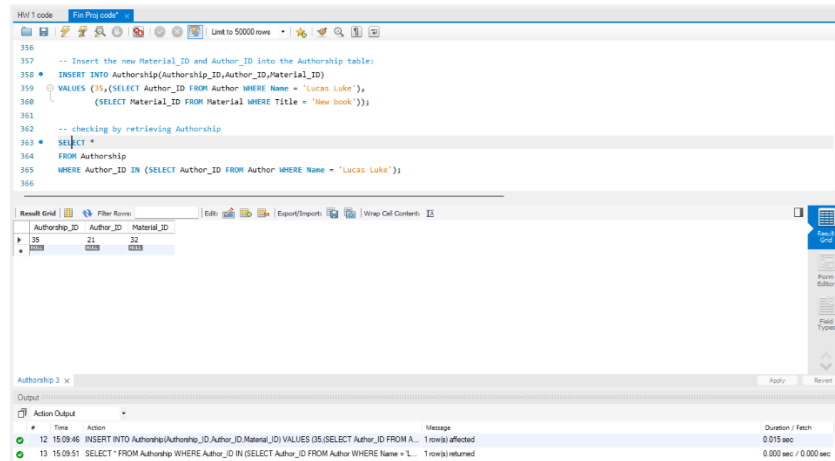
-- Insert the new Material_ID and Author_ID into the Authorship table:

```
INSERT INTO Authorship(Authorship_ID, Author_ID, Material_ID)
VALUES (35, (SELECT Author_ID FROM Author WHERE Name = 'Lucas Luke'),
(SELECT Material_ID FROM Material WHERE Title = 'New book'));
```

-- checking by retrieving Authorship

```
SELECT *
FROM Authorship
WHERE Author_ID IN (SELECT Author_ID FROM Author WHERE Name = 'Lucas
Luke');
```

Result:



4.2. Design

1. Alert staff about overdue materials on a daily basis?

Example of a SQL query, this might be use in your daily alerting process. This would need to schedule this query to run daily using an external scheduler or a script:

-- Example Query to Retrieve Overdue Materials using sql

```

SELECT
    Borrow.Member_ID,
    Member.Name AS MemberName,
    Material.Title AS MaterialTitle,
    Borrow.Due_Date
FROM
    Borrow
JOIN
    Material ON Borrow.Material_ID = Material.Material_ID
JOIN
    Member ON Borrow.Member_ID = Member.Member_ID
WHERE
    Borrow.Due_Date < CURRENT_DATE AND Borrow.Return_Date IS NULL;

```

Result:

Result Grid				
Filter Rows:		Export:	Wrap Cell Content: 15	
Member_ID	MemberName	MaterialTitle	Due_Date	
1	Alice Johnson	Frankenstein	2021-12-20	
9	Isla Thomas	The Catcher in the Rye	2023-01-18	
1	Alice Johnson	To Kill a Mockingbird	2023-02-13	
11	Kate Anderson	The Hobbit	2023-03-22	
12	Luke Jackson	The Shining	2023-03-31	
13	Mia White	Pride and Prejudice	2023-04-05	
17	Quinn Hall	The Great Gatsby	2023-04-15	
8	Harry Garcia	Moby Dick	2023-04-20	
9	Isla Thomas	Crime and Punishment	2023-04-16	
20	Tiffany Allen	The Hitchhiker's Guide to the Galaxy	2023-04-18	

Result 2 x				
Output				
Action Output				
#	Time	Action		Message
4	15:40:41	CREATE TABLE OverdueFees (FeeID SERIAL PRIMARY KEY, MemberID INT, Amount DECIMAL...		0 row(s) affected
5	15:44:13	SELECT Borrow.Member_ID, Member.Name AS MemberName, Material.Title AS MaterialTitle, B...		10 row(s) returned

In a real-world scenario, integrate this query into a script (e.g., using a programming language like Python or a shell script) that connects to the database, runs this query, and sends alerts to the staff based on the results.

2. Automatically deactivate the membership based on the member's overdue occurrence (>= three times). And reactivate the membership once the member pays the overdue fee.

To automate membership deactivation based on overdue occurrences and reactivation upon payment, need to modify the database schema and create some additional tables to track membership status and overdue fees.

Automation Process:

Automate membership deactivation and reactivation:

- Create a script or program that runs daily to check for overdue materials, updates the Overdue Fees table for each member with the overdue amount, and tracks the number of overdue occurrences.
- If a member exceeds a certain number of overdue occurrences (e.g., three times), update the Status column in the Member table to "Deactivated."
- When a member pays the overdue fee, update the OverdueFees table to mark the fee as "Paid." This should be done by library staff when the payment is received.
- Implement a script to reactivate the membership by changing the Status column back to "Active" when the overdue fee is marked as "Paid."
- By incorporating these modifications and automation processes, you can automatically manage membership status based on overdue occurrences and payments within your library management system.